

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Computação Gráfica

Ano Letivo de 2025/2026

Fase 3: Curvas, Superfícies Cúbicas e VBOs

Carlos Araújo
A106845

João Teixeira
A106836

Simão Mendes
A106928

Vitor Senra
A106921

April 14, 2026

CG

Índice

1. Introdução	1
2. Geração de Superfícies de Bézier	2
2.1. Fundamentos Matemáticos	2
2.2. Algoritmo de Tesselação	3
3. Otimização do Motor com VBOs	6
3.1. Estruturas de Dados e Gestão de Memória	6
3.2. O Novo Processo de Renderização	7
3.3. Análise de Desempenho (<i>Benchmark</i>)	7
4. Animações Dinâmicas e Curvas de Catmull-Rom	9
4.1. Curvas de Catmull-Rom	9
4.1.1. Fundamentos Matemáticos	9
4.1.2. Equação Matricial	9
4.1.3. Cálculo para Múltiplas Coordenadas	10
4.2. Extensão para Curvas Fechadas com Parâmetro Global	10
4.2.1. Parametrização Global	10
4.2.2. Envolvimento Cíclico	11
4.3. Mapeamento de Tempo para Transformações	11
4.3.1. Parametrização Temporal	11
4.3.2. Translação Dinâmica	11
4.3.3. Rotação Dinâmica	12
4.4. Cálculo de Derivadas e Alinhamento de Modelos	12
4.4.1. Tangente da Curva	12
4.4.2. Normalização da Tangente	12
4.4.3. Construção do Quadro Local (<i>Frame de Frenet</i>)	12
4.4.4. Matriz de Rotação Resultante	13
4.5. Pipeline Completo de Animação	14
4.5.1. Fluxo de Transformação	14
4.5.2. Composição de Transformações	14
4.5.3. Continuidade Visual	14
4.6. Exemplos de Configuração	14
5. Resultados	16
5.1. Sistema Solar Dinâmico e Cometa	16
5.2. Teste 1: Animação e Curvas	16
5.3. Teste 2: Superfícies Cúbicas (<i>Patches de Bezier</i>)	17
6. Funcionalidades Extra	18
6.1. Abordagem e Algoritmo	18
6.2. Equações Paramétricas	18
6.3. Resultado Visual	19

7. Conclusão	20
8. Referências Bibliográficas	21

1. Introdução

Este relatório descreve o desenvolvimento e a implementação da terceira fase do projeto da unidade curricular de Computação Gráfica. O objetivo principal desta etapa consistiu em melhorar o motor de renderização (*engine*) e o gerador de modelos (*generator*) com novas capacidades, nomeadamente a geração de superfícies curvas complexas e a animação dinâmica de objetos no espaço 3D.

Ao longo desta fase, foram explorados quatro conceitos fundamentais:

- **Superfícies de Bézier:** Implementação no gerador da capacidade de criar malhas de triângulos a partir de *patches* de Bézier, permitindo a modelação de superfícies suaves como o conhecido “*Teapot*” de Utah.
- **Curvas de Catmull-Rom:** Extensão das transformações geométricas no motor para suportar translações dinâmicas ao longo de curvas suaves, garantindo que o movimento dos objetos seja fluído e natural.
- **Vertex Buffer Objects (VBOs):** Otimização da performance do motor através da migração do modo imediato (*Immediate Mode*) para o uso de VBOs, permitindo o armazenamento e processamento eficiente dos vértices diretamente na placa gráfica (GPU).
- **Novos Modelos:** Criação de um novo modelo 3D, o cometa, para enriquecer a diversidade visual do Sistema Solar.

Esta fase representa um salto significativo na complexidade visual e na eficiência do motor, permitindo a criação de cenas mais ricas e dinâmicas, como o sistema solar interativo demonstrado nos resultados.

2. Geração de Superfícies de *Bézier*

A introdução de superfícies curvas complexas representa uma evolução significativa na arquitetura do motor gráfico nesta fase do projeto. O gerador de geometria foi expandido para suportar a criação de modelos baseados em superfícies de *Bézier* (*patches*). A aplicação recebe agora como parâmetros de entrada um ficheiro de configuração contendo as coordenadas dos pontos de controlo, bem como o nível de tesselação desejado. O objetivo do processo algorítmico é processar esta informação para gerar uma lista de triângulos que defina com precisão a malha tridimensional correspondente.

2.1. Fundamentos Matemáticos

Uma superfície de *Bézier* bicúbica (um *patch*) é matematicamente definida por uma grelha de 4×4 pontos de controlo, totalizando 16 pontos. Qualquer coordenada $S(u, v)$ pertencente a esta superfície é calculada interpolando estes pontos de controlo através de dois parâmetros independentes, u e v , que variam ambos no intervalo contínuo $[0, 1]$.

A posição tridimensional de um dado ponto na superfície é obtida através do produto tensorial de duas curvas de *Bézier*, descrito pela seguinte equação paramétrica:

$$S(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i^3(u) B_j^3(v) P_{i,j}$$

Nesta equação, $P_{i,j}$ representa a matriz bidimensional com os 16 pontos de controlo do *patch*, enquanto B_i^3 e B_j^3 correspondem aos polinómios de Bernstein de grau 3. O polinómio de *Bernstein* genérico de grau n é definido matematicamente como:

$$B_i^{n(t)} = \binom{n}{i} t^i (1-t)^{n-i}$$

Para otimização teórica e compreensão computacional, a formulação da superfície pode também ser expressa através de operações matriciais:

$$S(u, v) = U \cdot M \cdot P \cdot M^T \cdot V^T$$

Onde $U = [u^3, u^2, u, 1]$ e $V = [v^3, v^2, v, 1]$ são os vetores de parâmetros, M é a matriz característica de *Bézier* (matriz base) e P a matriz 4×4 dos pontos de controlo associados àquele *patch* específico.

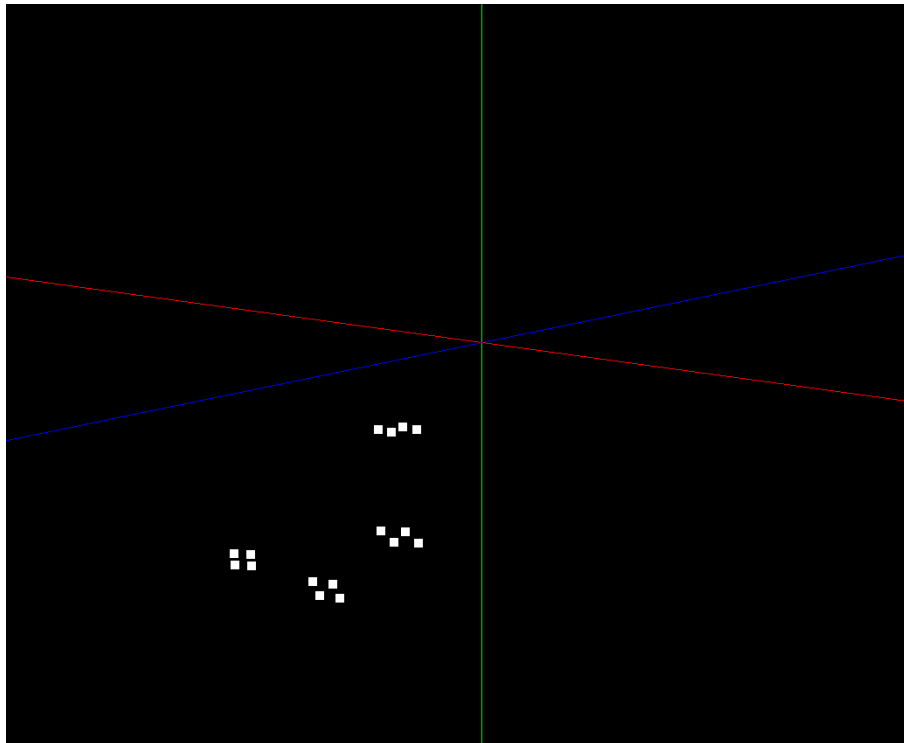


Figura 1: Representação esquemática dos 16 pontos de controlo que definem um *patch* de Bézier bicúbico.

2.2. Algoritmo de Tesselação

Os modelos geométricos em sistemas gráficos modernos (como o *OpenGL* suportado por *Vertex Buffer Objects*) não conseguem renderizar superfícies matemáticas contínuas de forma nativa. Deste modo, a superfície tem de ser discretizada numa malha de polígonos simples (triângulos), num processo denominado **tesselação**.

A quantidade de triângulos gerados e, conseqüentemente, a suavidade da curva resultante, são controladas pelo parâmetro de tesselação submetido pelo utilizador. O passo de incremento paramétrico (*step*) é simplesmente definido por $\frac{1}{\text{tesselação}}$.

O algoritmo processa a superfície iterando uma grelha bidimensional nos domínios u e v . Para cada quadrícula iterada, calculam-se os seus 4 vértices (utilizando a avaliação dos polinómios de *Bernstein* acima explicada) e geram-se dois triângulos adjacentes para preservar o correto *winding* (orientação dos vértices), garantindo que as normais (e futuramente a iluminação) serão calculadas de forma consistente na fase seguinte do projeto.

A lógica descrita é formalizada pelo seguinte pseudo-código:

```

1  Procedimento GerarSuperficieBezier(ficheiro,
    nivel_tesselacao): pseudocode
2      // 1. Leitura de dados
3      numero_patches <- LerFicheiro(ficheiro)
4      indices_patches <- LerIndices(ficheiro, numero_patches)

```

```

5     pontos_controlo <- LerPontosControlo(ficheiro)
6
7     // 2. Cálculo do incremento paramétrico
8     passo <- 1.0 / nivel_tesselacao
9     malha_triangulos <- NovaLista()
10
11    // 3. Avaliação e geração geométrica
12    Para cada patch em indices_patches:
13        Para i de 0 até nivel_tesselacao - 1:
14            Para j de 0 até nivel_tesselacao - 1:
15
16                // Determinação do domínio paramétrico [u, v] para a
17                // sub-grelha
18                u1 <- i * passo
19                v1 <- j * passo
20                u2 <- (i + 1) * passo
21                v2 <- (j + 1) * passo
22
23                // Cálculo das coordenadas 3D exatas na superfície
24                V1 <- AvaliarPontoBezier(u1, v1, patch,
25                pontos_controlo)
26                V2 <- AvaliarPontoBezier(u2, v1, patch,
27                pontos_controlo)
28                V3 <- AvaliarPontoBezier(u1, v2, patch,
29                pontos_controlo)
30                V4 <- AvaliarPontoBezier(u2, v2, patch,
31                pontos_controlo)
32
33                // Subdivisão da quadrícula em dois triângulos
34                // (respeitando a regra da mão direita)
35                AdicionarTriangulo(malha_triangulos, V1, V2, V3)
36                AdicionarTriangulo(malha_triangulos, V2, V4, V3)
37
38    Retornar malha_triangulos

```

Através da implementação deste algoritmo, o sistema adquire a capacidade de desenhar objetos de enorme complexidade orgânica, como demonstrado na recriação do “*Teapot*” de *Utah*, mantendo uma performance escalável face às exigências gráficas da cena de demonstração delineada.

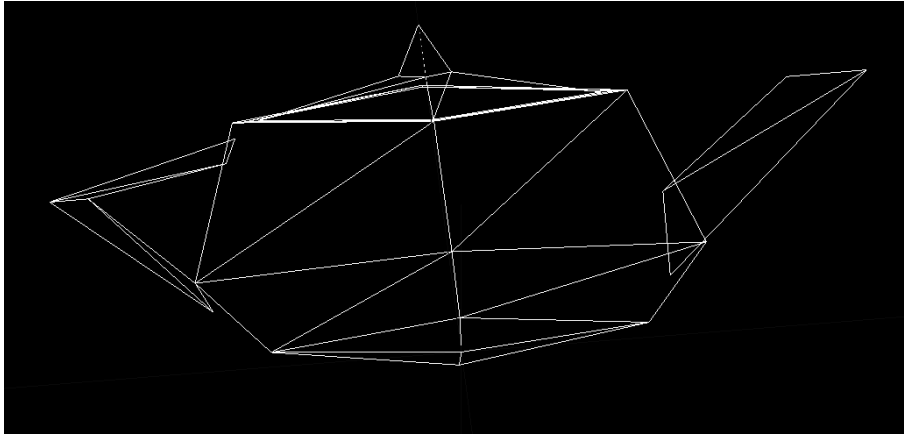


Figura 2: Representação em modo *wireframe* da tesselação básica (nível 1).

3. Otimização do Motor com VBOs

Nas fases anteriores do projeto, o desenho dos modelos era realizado através do modo imediato do *OpenGL*. Esta abordagem obrigava o processador (CPU) a percorrer as estruturas de dados e a enviar as coordenadas de cada vértice individualmente para a placa gráfica (GPU) a cada *frame* gerada.

Com o aumento substancial do número de vértices decorrente da modelação de superfícies complexas (como as de *Bézier* na Fase 3), este método revelou-se altamente ineficiente. A necessidade constante de transferir *megabytes* de dados continuamente, a cada ciclo de renderização, criou um estrangulamento (*bottleneck*) grave no barramento de comunicação entre o CPU e a Placa Gráfica, resultando em quedas drásticas de desempenho.

Modo Imediato (Fases 1 e 2)	Modo VBO (Fase 3)
Por cada <i>Frame</i> gerada: 1. CPU lê Vértice 1 → Envia para GPU 2. CPU lê Vértice 2 → Envia para GPU 3. CPU lê Vértice 3 → Envia para GPU <i>... repetido para todos os vértices ...</i> Resultado: Barramento congestionado, GPU ocioso à espera que os dados cheguem.	Fase de Inicialização (1 única vez): CPU envia o bloco completo de dados para a VRAM. Por cada <i>Frame</i> gerada: CPU envia apenas um sinal de desenho. GPU acede e lê da sua própria memória a alta velocidade.

Tabela 1: Comparação arquitetural entre a submissão de vértices no Modo Imediato e o uso de VBOs.

3.1. Estruturas de Dados e Gestão de Memória

Para mitigar o problema do barramento, a arquitetura interna do motor foi reestruturada para utilizar **Vertex Buffer Objects** (VBOs). Esta tecnologia permite que os dados geométricos sejam transferidos em bloco e armazenados permanentemente na memória de vídeo (VRAM).

Para suportar este novo paradigma, a classe responsável pelos modelos 3D sofreu as seguintes adaptações:

- **Arrays Contíguos:** Em vez de processar os pontos avulsos, os vértices lidos do ficheiro `.3d` passaram a ser agregados em **Arrays** contíguos na memória

RAM principal. Implementámos também um **Array de Índices**, permitindo o reaproveitamento de vértices partilhados por diferentes triângulos, o que reduz substancialmente o consumo global de memória.

- **Identificadores de Buffers:** A estrutura do modelo passou a incluir variáveis numéricas que funcionam como identificadores (IDs). Estes IDs são as referências que a Placa Gráfica devolve ao CPU para indicar a localização onde os dados foram guardados (um ID alocado para os Vértices e outro para os Índices).

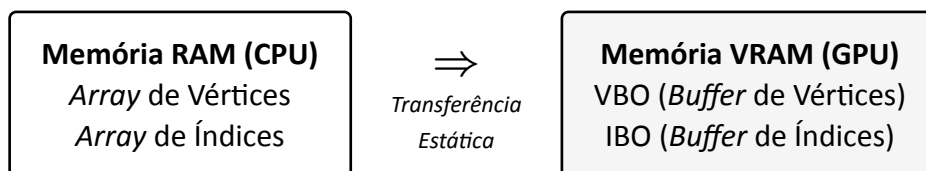


Figura 3: Fluxo do ciclo de vida da memória geométrica no novo motor.

3.2. O Novo Processo de Renderização

Com as novas estruturas estabelecidas, a lógica do motor foi formalmente dividida em duas etapas independentes:

1. **Fase de Preparação:** Executada uma única vez após a leitura dos ficheiros. O motor solicita a geração dos **buffers** no GPU, ativa-os e executa uma cópia integral dos dados geométricos da RAM para a VRAM. Informamos a API gráfica de que estes dados serão desenhados frequentemente, mas não sofrerão alterações matemáticas por parte do CPU, permitindo à Placa Gráfica otimizar fisicamente a sua localização interna.
2. **Fase de Desenho:** Executada continuamente no ciclo principal do motor. Durante a renderização, o CPU deixa de enviar coordenadas. A sua única responsabilidade passa por ativar os identificadores (IDs) corretos e instruir a Placa Gráfica sobre o formato da informação que lá está guardada (espaçamento e tipo de dados). De seguida, emite uma única ordem de desenho indexado em bloco, delegando o trabalho massivo de triangulação puramente para o processador gráfico.

Esta delegação estrutural eliminou o gargalo de transferência, resultando numa capacidade de renderização de cenários consideravelmente mais densos e poligonais, mantendo uma taxa de *frames* por segundo (FPS) muito mais elevada e estável.

3.3. Análise de Desempenho (*Benchmark*)

Para comprovar a eficácia da otimização, realizámos testes comparativos de FPS com o **V-Sync** desativado, utilizando a geometria da Fase 2 como base de teste estável.

Cenário / Modelo	Nº Vértices	FPS (Imediato)	FPS (VBOs)	Ganho
Esfera (20x20)	2.280	6.800	8.200	+20.6%
Anel de Saturno (80x80)	38.400	3.500	7.800	+122.9%
Sistema Solar Completo	85.000	780	4.400	+464.1%

Tabela 2: Comparativo de desempenho entre arquiteturas de transferência de memória.

Os resultados demonstram que, mesmo em máquinas de alto desempenho, o Modo Imediato limita severamente a escalabilidade do motor à medida que a complexidade da cena aumenta. Enquanto na esfera simples o ganho é marginal, no Sistema Solar Completo verificou-se uma melhoria de rendimento superior a 400%. A introdução de VBOs eliminou efetivamente o gargalo de transferência, permitindo taxas de *frames* por segundo (FPS) significativamente superiores e maior estabilidade em ambientes poligonais densos.

4. Animações Dinâmicas e Curvas de *Catmull-Rom*

As animações dinâmicas são fundamentais em aplicações de computação gráfica, permitindo o movimento suave de objetos ao longo de caminhos predefinidos. Neste capítulo, exploramos a implementação de curvas de *Catmull-Rom* para gerar trajetórias contínuas e suaves, bem como o mecanismo de alinhamento automático de modelos através de cálculos de derivadas.

O sistema de animação desenvolvido utiliza três componentes principais:

1. Curvas de *Catmull-Rom* para definir trajetórias suaves
2. Mapeamento de tempo em transformações (translação e rotação)
3. Cálculo de derivadas para orientação automática de modelos

4.1. Curvas de *Catmull-Rom*

4.1.1. Fundamentos Matemáticos

A curva de *Catmull-Rom* é uma *spline* cúbica paramétrica que passa através de pontos de controle intermediários. Diferentemente de outras *splines*, como a *Bézier*, a *Catmull-Rom* garante continuidade geométrica através de múltiplos segmentos e passa pelos pontos de controle especificados.

Uma curva de *Catmull-Rom* é definida por quatro pontos de controle: p_0, p_1, p_2, p_3 . A curva passa pelos pontos intermediários p_1 e p_2 , enquanto p_0 e p_3 influenciam a curvatura da trajetória.

4.1.2. Equação Matricial

A posição em um segmento de *Catmull-Rom* para um parâmetro $t \in [0, 1]$ é dado por:

$$p(t) = T(t) \cdot M_{CR} \cdot P$$

Onde:

- $T(t) = [t^3, t^2, t, 1]$ é o vetor de base temporal
- M_{CR} é a matriz de *Catmull-Rom*
- $P = [p_0, p_1, p_2, p_3]^T$ é o vetor de pontos de controle

A matriz de *Catmull-Rom* é definida como:

$$M_{CR} = \frac{1}{2} \begin{bmatrix} -1 & 3 & -3 & 1 \\ 2 & -5 & 4 & -1 \\ -1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0 \end{bmatrix}$$

Portanto, a posição é calculada como:

$$p(t) = \frac{1}{2}[t^3, t^2, t, 1] \cdot \begin{bmatrix} -1 & 3 & -3 & 1 \\ 2 & -5 & 4 & -1 \\ -1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix}$$

Simplificando através da multiplicação matricial, obtemos os coeficientes:

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = M_{CR} \cdot \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix}$$

E a posição final é expressa como um polinómio cúbico:

$$p(t) = a_0 t^3 + a_1 t^2 + a_2 t + a_3$$

4.1.3. Cálculo para Múltiplas Coordenadas

Como as coordenadas x , y e z são independentes, o cálculo é realizado separadamente para cada eixo. Para a coordenada x :

$$x(t) = a_{\{0,x\}} t^3 + a_{\{1,x\}} t^2 + a_{\{2,x\}} t + a_{\{3,x\}}$$

Onde:

$$\begin{bmatrix} a_{\{0,x\}} \\ a_{\{1,x\}} \\ a_{\{2,x\}} \\ a_{\{3,x\}} \end{bmatrix} = M_{CR} \cdot \begin{bmatrix} p_{\{0,x\}} \\ p_{\{1,x\}} \\ p_{\{2,x\}} \\ p_{\{3,x\}} \end{bmatrix}$$

As mesmas operações são repetidas para as coordenadas y e z , resultando em um vetor de posição 3D:

$$p(t) = [x(t), y(t), z(t)]$$

4.2. Extensão para Curvas Fechadas com Parâmetro Global

4.2.1. Parametrização Global

Para animar um objeto ao longo de um caminho completo formado por múltiplos segmentos de *Catmull-Rom*, utiliza-se um parâmetro global $g_t \in [0, 1]$ que representa a progressão ao longo de toda a curva.

Dado um conjunto de n pontos de controlo, o parâmetro global é mapeado para um segmento específico através de:

$$t_{\text{real}} = g_t \times n$$

O índice do segmento é determinado por:

$$s = \lfloor t_{\text{real}} \rfloor$$

E o parâmetro local dentro do segmento é:

$$t_{\text{local}} = t_{\text{real}} - s$$

4.2.2. Envolvimento Cíclico

Para criar curvas fechadas (*loops*), os índices são calculados com envolvimento cíclico usando a operação módulo:

$$p_0 = \text{controlPoints}[(s - 1 + n) \bmod n]$$

$$p_1 = \text{controlPoints}[s \bmod n]$$

$$p_2 = \text{controlPoints}[(s + 1) \bmod n]$$

$$p_3 = \text{controlPoints}[(s + 2) \bmod n]$$

Isso garante que a curva seja contínua e suave, mesmo ao retornar ao ponto inicial.

4.3. Mapeamento de Tempo para Transformações

4.3.1. Parametrização Temporal

As animações são controladas pelo tempo decorrido desde o início da aplicação. O tempo decorrido em segundos é mapeado para um parâmetro normalizado $g_t \in [0, 1]$ através de uma função periódica com período T (duração da animação em segundos):

$$g_t = \frac{\text{mod}(t_{\text{decorrido}}, T)}{T}$$

Onde $\text{mod}(t_{\text{decorrido}}, T)$ garante que a animação se repita indefinidamente após completar um ciclo.

4.3.2. Translação Dinâmica

Para objetos em movimento ao longo de uma curva de *Catmull-Rom*, a posição em cada *frame* é calculada através de:

$$p(t) = \text{getGlobalCatmullRomPoint}(g_t, \text{controlPoints})$$

O objeto é então transladado para essa posição utilizando:

$$T = \begin{bmatrix} 1 & 0 & 0 & p_x(t) \\ 0 & 1 & 0 & p_y(t) \\ 0 & 0 & 1 & p_z(t) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.3.3. Rotação Dinâmica

Para rotações contínuas ao redor de um eixo específico, o ângulo é calculado proporcionalmente ao tempo decorrido:

$$\theta(t) = \frac{\text{mod}(t_{\text{decorrido}}, T)}{T} \times 360^\circ$$

A matriz de rotação ao redor de um eixo $\mathbf{a} = [a_x, a_y, a_z]$ (normalizado) é dada pela fórmula de Rodrigues:

$$\mathbf{R}(\theta) = \cos(\theta)\mathbf{I} + \sin(\theta)[\mathbf{a}]_{\times} + (1 - \cos(\theta))\mathbf{a}\mathbf{a}^T$$

Onde $[\mathbf{a}]_{\times}$ é a matriz anti-simétrica do vetor $\mathbf{a}$.

4.4. Cálculo de Derivadas e Alinhamento de Modelos

4.4.1. Tangente da Curva

A derivada da função de posição fornece o vetor tangente à curva, que indica a direção do movimento:

$$\mathbf{p}'(t) = \frac{d}{dt}[\mathbf{p}(t)] = 3a_0t^2 + 2a_1t + a_2$$

Novamente, este cálculo é realizado para cada coordenada independentemente:

$$\mathbf{p}'(t) = [x'(t), y'(t), z'(t)]$$

Este vetor tangente tem grande significância: sua magnitude indica a velocidade local ao longo da curva, e sua direção aponta para a qual o objeto deveria estar orientado.

4.4.2. Normalização da Tangente

O vetor tangente é normalizado para obter um vetor unitário que representa apenas a direção:

$$\mathbf{T} = \frac{\mathbf{p}'(t)}{\|\mathbf{p}'(t)\|}$$

Onde a norma euclidiana é:

$$\|\mathbf{p}'(t)\| = \sqrt{x'(t)^2 + y'(t)^2 + z'(t)^2}$$

4.4.3. Construção do Quadro Local (*Frame de Frenet*)

Para alinhar um modelo com a orientação da curva, é necessário construir um quadro de referência local (sistema de coordenadas) que inclua:

- **Eixo T (Tangente):** Direção do movimento, alinhado com $\mathbf{T}$
- **Eixo N (Normal):** Perpendicular ao eixo T, aponta para o “centro” da curvatura

- **Eixo B (Binormal):** Perpendicular a ambos T e N, define o “cima” do objeto

O quadro de *Frenet* clássico requer cálculos adicionais. Contudo, uma abordagem prática usa um vetor “up” de referência para construir uma base ortonormal:

Passo 1: Definir o Eixo X (Tangente)

$$X = T = \frac{p'(t)}{\|p'(t)\|}$$

Passo 2: Definir o Eixo Z (via produto vetorial)

Usa-se um vetor “up” de referência (frequentemente $[0, 1, 0]$ ou o valor anterior) e calcula-se:

$$Z = X \times Y_{\text{prev}}$$

Onde Y_{prev} é o eixo Y calculado no *frame* anterior, garantindo continuidade visual. Normaliza-se:

$$Z = \frac{Z}{\|Z\|}$$

Passo 3: Recalcular o Eixo Y (Normal)

Para garantir ortogonalidade:

$$Y = Z \times X$$

Normalizado:

$$Y = \frac{Y}{\|Y\|}$$

4.4.4. Matriz de Rotação Resultante

Os três eixos ortonormais formam uma matriz de rotação 4x4:

$$R = \begin{bmatrix} X_x & Y_x & Z_x & 0 \\ X_y & Y_y & Z_y & 0 \\ X_z & Y_z & Z_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Ou em notação mais compacta:

$$R = [X \mid Y \mid Z \mid 0]^T$$

Esta matriz quando aplicada ao objeto causa uma rotação tal que o modelo “segue” a orientação da curva.

4.5. Pipeline Completo de Animação

4.5.1. Fluxo de Transformação

O sistema de animação funciona através do seguinte pipeline a cada frame:

1. **Captura de Tempo:** Obter $t_{\text{decorrido}}$ do relógio do sistema.
2. **Parametrização:** Calcular $g_t = \frac{\text{mod}(t_{\text{decorrido}}, T)}{T}$.
3. **Avaliação da Curva:** Chamar `getGlobalCatmullRomPoint(g_t, ...)` obtendo:
 - Posição $p(t)$.
 - Derivada $p'(t)$.
4. **Construção de Transformação:** Montar matrizes de translação e rotação.
5. **Aplicação:** Aplicar transformações à pilha de matrizes do *OpenGL*.
6. **Renderização:** Desenhar o modelo transformado.

4.5.2. Composição de Transformações

Quando múltiplas transformações são aplicadas (translação + rotação com alinhamento), a ordem é crucial:

$$M_{\text{final}} = M_{\text{translacao}} \times R_{\text{alinhamento}} \times R_{\text{rotacao}}$$

Isso garante que o objeto é primeiro rotacionado para se alinhar com a curva, depois transladado ao longo da trajetória.

4.5.3. Continuidade Visual

A continuidade visual é garantida através de:

1. **Interpolação suave:** A *spline Catmull-Rom* garante continuidade C^2 (primeira e segunda derivadas contínuas)
2. **Consistência de *frame*:** O vetor Y_{prev} mantido entre *frames* assegura que não há “rotações extras” inesperadas
3. **Envolvimento cíclico:** Pontos de controle são acessados com módulo, criando transições suaves no retorno ao início

4.6. Exemplos de Configuração

Uma animação típica é configurada via XML com a seguinte estrutura:

```
1 <translate time="10" align="true">
2   <point x="0" y="0" z="4" />
3   <point x="4" y="0" z="0" />
4   <point x="0" y="0" z="-4" />
5   <point x="-4" y="10" z="0" />
6 </translate>
```

Neste exemplo:

- `time="10"`: A animação dura 10 segundos, após o qual retorna ao início.
- `align="true"`: O modelo será rotacionado para alinhar-se com a curva.
- Os 4 `<point>` elementos definem os pontos de controle da *spline*.

O período $T = 10$ segundos, e o objeto viaja suavemente pela curva de *Catmull-Rom* formada por estes 4 pontos, com rotação automática.

5. Resultados

Neste capítulo são apresentados os resultados visuais das implementações desenvolvidas durante a Fase 3 do projeto, nomeadamente a evolução do sistema solar para um ambiente dinâmico e a validação do motor 3D através dos ficheiros de teste fornecidos.

5.1. Sistema Solar Dinâmico e Cometa

Nesta fase, o modelo estático do sistema solar foi atualizado para incluir órbitas dependentes do tempo e um cometa cuja trajetória é definida por uma curva de *Catmull-Rom*.

Devido à natureza dinâmica desta implementação, a demonstração completa foi registada em vídeo. A figura abaixo ilustra a progressão temporal da cena através de fotogramas-chave (*keyframes*), onde é possível observar o movimento do cometa criado com *patches* de *Bézier* ao longo da sua órbita.

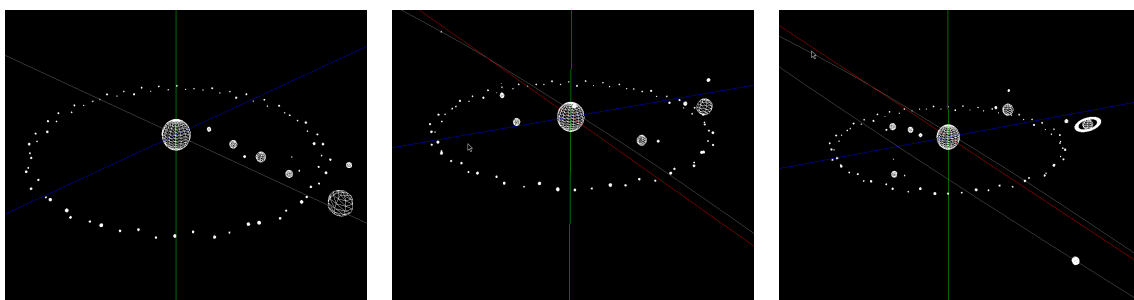


Figura 4: Sequência temporal do Sistema Solar Dinâmico.

Vídeo demonstrativo completo disponível em: [Clique aqui para aceder ao vídeo](#).

5.2. Teste 1: Animação e Curvas

O primeiro teste fornecido pelos docentes foca-se na translação temporal e na orientação baseada na curva (alinhamento). Tal como no sistema solar, o resultado é um comportamento contínuo no tempo. Abaixo apresenta-se a captura dos momentos cruciais da animação.

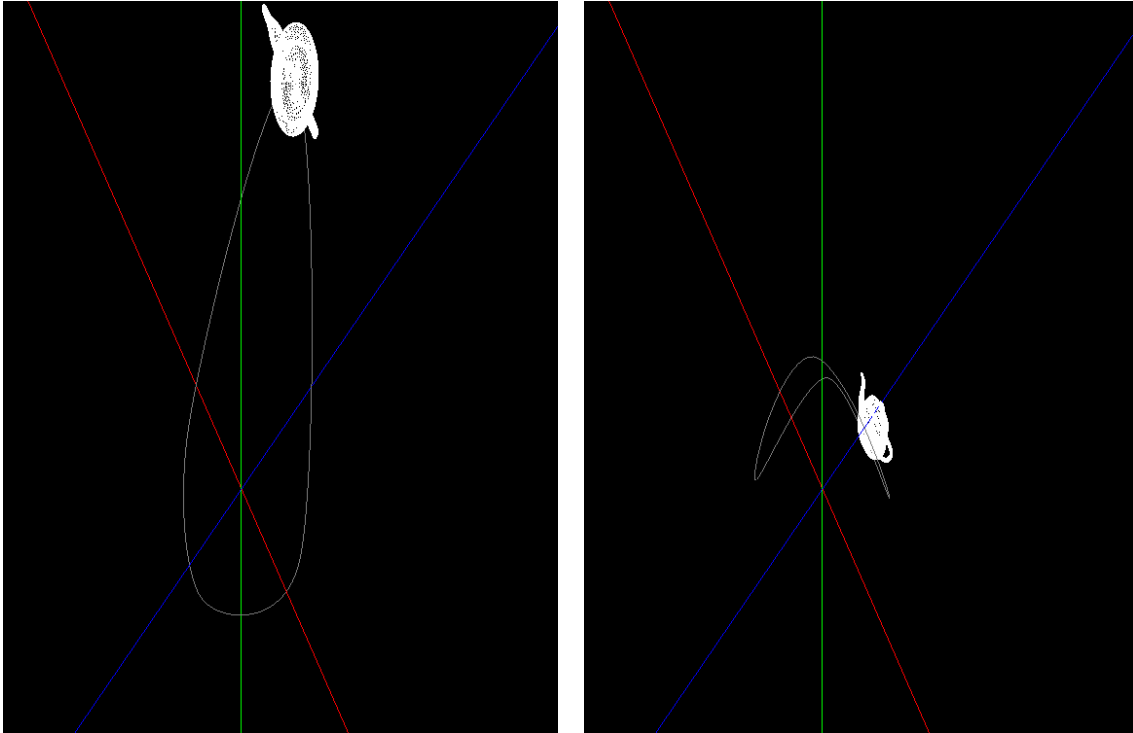


Figura 5: Fotografamas do Teste 1, evidenciando o comportamento da curva de *Catmull-Rom* e VBOs.

Vídeo do Teste 1 disponível em: [Aceder ao vídeo](#).

5.3. Teste 2: Superfícies Cúbicas (*Patches de Bezier*)

O segundo ficheiro de teste valida a geração de modelos a partir de *patches* de Bézier (como o bule de chá - *teapot*) e a sua renderização com *Vertex Buffer Objects* (VBOs). A imagem abaixo demonstra o resultado estático obtido, que coincide com a saída visual esperada.

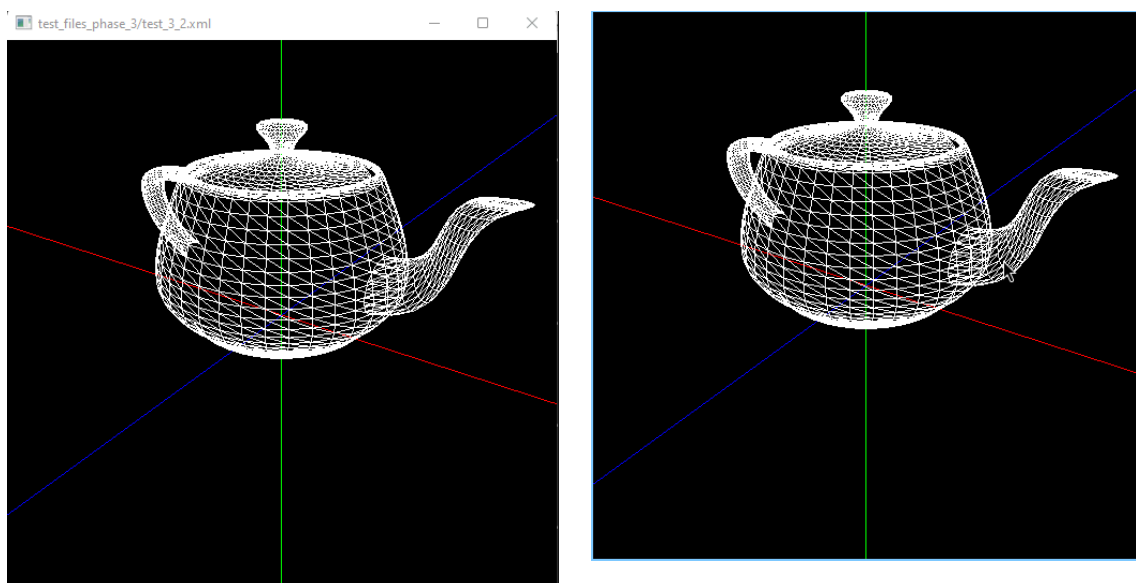


Figura 6: Resultado visual do Teste 2: Professores (esquerda) vs. Nosso (direita).

6. Funcionalidades Extra

Para enriquecer a simulação do sistema solar requerida na Fase 3 do projeto, foi concebida e desenvolvida uma nova primitiva geométrica que representa um corpo celeste irregular: um cometa. Esta implementação enquadra-se na componente de valorização (Extras) do motor 3D.

6.1. Abordagem e Algoritmo

O objetivo principal desta primitiva foi criar um modelo procedural que se assemelhasse a um asteroide ou cometa, evitando a perfeição geométrica de uma esfera tradicional. O algoritmo desenvolvido baseia-se na topologia de uma esfera (parametrizada por raio, fatias e pilhas), aplicando-lhe uma deformação estocástica (ruído) em cada vértice.

A aplicação de ruído estocástico a uma malha topológica apresenta o risco de “rasgar” a superfície, o que ocorre se vértices partilhados por triângulos adjacentes sofrerem deformações distintas. Para garantir que a malha do cometa permanece uma superfície perfeitamente fechada, foram implementados três mecanismos de controlo na atribuição dos raios:

1. **Caching de Deformações:** Utilizou-se uma estrutura de dados baseada em dicionário (`std::map`) que associa cada coordenada discreta (i, j) da grelha ao seu raio deformado. Quando fatias ou pilhas adjacentes acedem ao mesmo ponto lógico, o algoritmo devolve o valor em memória em vez de recalcular o ruído, assegurando a ligação dos vértices no espaço tridimensional.
2. **Costura Lateral (*Wrap-around*):** Para fechar a forma em 360 graus, a última fatia gerada tem de coincidir exatamente com a primeira. Isto foi resolvido aplicando uma operação de módulo ao índice da fatia. Desta forma, os vértices que fecham o ciclo reutilizam deterministicamente as deformações dos vértices de abertura.
3. **Convergência Polar:** Nos polos topológicos ($j = 0$ e $j = \text{stacks}$), múltiplas fatias convergem para uma única coordenada espacial real. Para evitar que o topo e a base do cometa apresentem múltiplas deformações (o que criaria falhas geométricas), o algoritmo anula a dimensão i nestes casos, guardando e acedendo a um único raio partilhado, associado à coordenada singular $(-1, j)$.

6.2. Equações Paramétricas

A projeção dos vértices para o espaço cartesiano 3D é feita através da conversão de coordenadas esféricas, utilizando o raio deformado especificamente calculado para cada ponto. Sendo α o ângulo da fatia atual e β o ângulo da pilha atual, as equações utilizadas foram:

$$x = r_{i,j} \cdot \sin(\beta) \cdot \sin(\alpha)$$

$$y = r_{i,j} \cdot \cos(\beta)$$

$$z = r_{i,j} \cdot \sin(\beta) \cdot \cos(\alpha)$$

6.3. Resultado Visual

Abaixo encontra-se a demonstração visual do cometa gerado e integrado no motor 3D, ilustrando a sua superfície irregular.

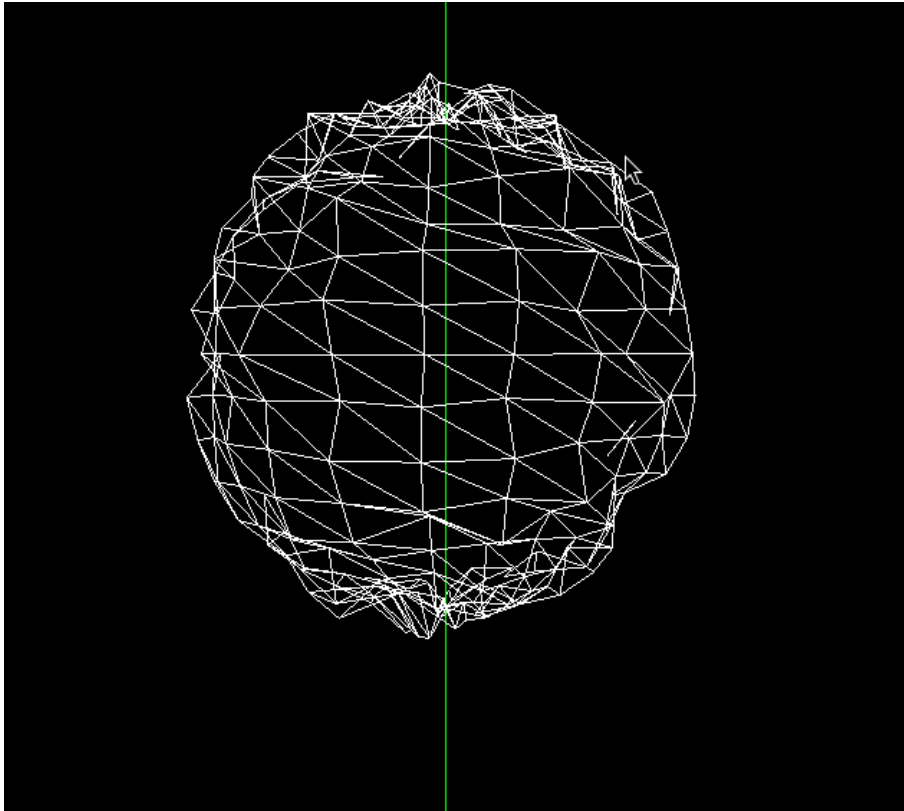


Figura 7: Renderização da primitiva “Cometa”, ilustrando a deformação processual da superfície.

7. Conclusão

Em conclusão, a terceira fase do projeto representou um marco fundamental no desenvolvimento do motor gráfico. Com a implementação das superfícies de *Bézier* e das curvas de *Catmull-Rom*, o motor deixou de estar limitado a primitivas geométricas básicas e transformações estáticas, passando a ser capaz de representar formas e movimentos complexos com um elevado grau de realismo visual.

A migração do modo imediato para o uso de *Vertex Buffer Objects (VBOs)* foi a alteração técnica mais profunda no núcleo do motor. Esta otimização não só melhorou significativamente o desempenho e a gestão de memória, como também preparou a base tecnológica para as próximas fases do projeto, onde a inclusão de iluminação e texturas exigirá um processamento ainda mais intensivo por parte da placa gráfica.

No geral, os objetivos propostos para esta fase foram plenamente atingidos. O sistema solar dinâmico criado serve como prova de conceito para a robustez das novas funcionalidades, demonstrando a correta integração entre a modelação matemática de curvas e superfícies e a sua representação visual em tempo real. Os conhecimentos adquiridos sobre o *pipeline* gráfico moderno serão essenciais para concluir o projeto com sucesso nas fases subsequentes.

8. Referências Bibliográficas

- [1] D. Shreiner, M. Woo, J. Neider, e T. Davis, *OpenGL Programming Guide*, 8.º ed. Addison Wesley, 2013.
- [2] L. Thomason, «tinyxml2». [Em linha]. Disponível em: <https://github.com/leethomason/tinyxml2>